

Intégration numérique d'une équation différentielle

7 octobre 2020

1 Intégrateurs à pas adaptatif

On présente ici une famille de méthodes d'intégration des équations différentielles plus versatile gérant elle-même le choix du pas de temps d'intégration h .

L'utilisateur doit d'abord bien connaître la physique du problème qu'il traite car il doit définir une valeur acceptable ϵ_m de l'erreur d'intégration à chaque pas.

A chaque itération, l'idée principale est d'ajuster le pas d'intégration de manière à ce que l'erreur ϵ ne dépasse pas la valeur de l'erreur acceptable ϵ_m . L'algorithme consiste à effectuer l'intégration du système d'équations différentielles de l'instant t_n à l'instant $t_n + h$, deux fois avec deux méthodes différentes, la seconde étant d'un ordre de précision supérieur à la première.

Chacune donne une estimation de $Y(t_n + h)$ que l'on note A_1 et A_2 qui nous serviront à calculer une estimation de l'erreur commise lors de l'intégration.

1.1 Méthode d'Euler à 2 pas

Pour notre démonstration, on suppose connaître la fonction $\Phi(t)$, solution analytique de notre système d'équations différentielles $\frac{dY}{dt} = f(t, Y)$.

Sans perdre en généralité, on suppose qu'à l'instant n , les solutions analytique et numérique coïncident : $\phi(t_n) \equiv Y_n$. Dans la phase 1 de l'algorithme, on applique la méthode d'Euler d'ordre 1 avec un pas d'intégration h . On obtient :

$$A_1 = Y_n + h f(t_n, Y_n) \quad (1)$$

or, après développement et substitution au voisinage de t_n , on a :

$$\begin{aligned} \phi(t_n + h) &= \phi(t_n) + h \phi'(t_n) + \frac{h^2}{2} \phi''(t_n) + \vartheta(h^3) \\ &= Y_n + h f(t_n, Y_n) + \frac{h^2}{2} \phi''(t_n) + \vartheta(h^3) \end{aligned} \quad (2)$$

Ceci implique que

$$A_1 = \phi(t_n + h) - \frac{h^2}{2} \phi''(t_n) + \vartheta(h^3) \quad (3)$$

Dans la phase 2, on applique la méthode d'Euler améliorée (autrement dit RK2), on sait que :

$$A_2 = \phi(t_n + h) + \vartheta(h^3) \quad (4)$$

Par différence des estimations A_1 et A_2 , on déduit l'erreur d'intégration :

$$\epsilon = |A_1 - A_2| = \frac{h^2}{2} |\phi''(t_n)| \quad (5)$$

On obtient l'algorithme de principe suivant :

Si $\epsilon > \epsilon_m$ alors
 l'itération est rejetée
 On diminue le pas de temps selon

$$h' = \sqrt{\frac{\epsilon_m}{\epsilon}}.h \quad A_1 = \phi(t_n + h) - \frac{h^2}{2} \phi''(t_n) + \mathcal{O}(h^3)$$

 Sinon
 l'itération est validée

$$y_{n+1} = A_2 + \mathcal{O}(h^3)$$

 On augmente le pas de temps selon

$$h' = \sqrt{\frac{\epsilon_m}{\epsilon}}.h$$

 Finsi

FIGURE 1 – Algorithme de la méthode d'Euler à 2 pas

Une implémentation de l'algorithme en Matlab vous est présentée ci-dessous :

Listing 1 – Implémentation de l'algorithme

```
% rk_adapt12(@equadif,[ti tf dt],[y1i y2i y3i], options);  
% methode d'Euler a pas adaptatif  
%  
5 function [tvec, yvec]=rk_adapt12(f, param, y0, options)  
AbsTol=1e-6;  
% intervalle d'etude en temps et pas  
t0 = param(1);  
tf = param(2);  
10 h = param(3);  
  
% sauvegarde du vecteur y0 dans la 1ere ligne du tableau yvec  
n=1;  
tvec(n) = t0;  
15 yvec(n, :) = y0;  
  
t = t0;  
y = y0;  
  
20 while (t<tf)  
    f1=f(t, y);  
    f2=f(t+h/2., y+h/2.*k1);  
  
    A1=y + h*f1;    % RK1  
25    A2=y + h*f2;    % RK2  
  
    E=A1-A2;  
    norm_E=norm(E, inf);  
    h=0.9*sqrt(AbsTol/norm_E)*h;  
30  
    if (norm_E <= AbsTol)  
        % pas suivant  
        t=t+h;  
        n=n+1;  
35        y=A2;  
  
    % stockage  
        tvec(n)=t;  
        yvec(n, :)=y;  
40 end  
end
```

1.2 Méthode de Kutta -Merson

$$\left\{ \begin{array}{l} K_1 = h f(y_n, t_n) \\ K_2 = h f\left(t_n + \frac{h}{3}, y_n + \frac{K_1}{3}\right) \\ K_3 = h f\left(t_n + \frac{h}{3}, y_n + \frac{K_1}{6} + \frac{K_2}{6}\right) \\ K_4 = h f\left(t_n + \frac{h}{2}, y_n + \frac{K_1}{8} + \frac{3K_3}{8}\right) \\ K_5 = h f\left(t_n + h, y_n + \frac{K_1}{2} - \frac{3K_3}{2} + 2K_4\right) \end{array} \right. \quad et \quad \left\{ \begin{array}{l} A_1 = y_n + \frac{K_1}{2} - \frac{3K_3}{2} + 2K_4 \\ \quad = \phi(t_n + h) - \frac{h^5}{120} \phi^{(5)}(t_n) + \mathcal{O}(h^6) \\ A_2 = y_n + \frac{K_1}{6} + \frac{2K_4}{3} + \frac{K_5}{6} \\ \quad = \phi(t_n + h) - \frac{h^5}{720} \phi^{(5)}(t_n) + \mathcal{O}(h^6) \end{array} \right.$$

C'est une méthode d'intégration d'ordre 5

L'erreur d'intégration à chaque pas peut être estimée



$$A_1 - A_2 = -\frac{5h^2}{720} \phi^{(5)}(t_n) + \mathcal{O}(h^6) \quad \varepsilon = \frac{-h^5}{720} \phi^{(5)}(t_n) = \frac{1}{5} (A_1 - A_2)$$

FIGURE 2 – Principe de la méthode de Kutta -Merson

A chaque pas, l'erreur $\varepsilon = \frac{1}{5} (A_1 - A_2)$ est calculée

Si $\varepsilon > \varepsilon_m$ alors

l'itération est rejetée

On diminue le pas de temps selon

$$h' = \left(\frac{\varepsilon_m}{\varepsilon} \right)^{1/5} h$$

Sinon

l'itération est validée

$$y_{n+1} = A_2 - \varepsilon + \mathcal{O}(h^6)$$

On augmente le pas de temps selon

$$h' = \left(\frac{\varepsilon_m}{\varepsilon} \right)^{1/5} h$$

Finsi

$$\left\{ \begin{array}{l} A_1 = \phi(t_n + h) - \frac{h^5}{120} \phi^{(5)}(t_n) + \mathcal{O}(h^6) \\ A_2 = \phi(t_n + h) - \frac{h^5}{720} \phi^{(5)}(t_n) + \mathcal{O}(h^6) \end{array} \right.$$

FIGURE 3 – Algorithme de la méthode de Kutta -Merson

Listing 2 – Implémentation de la méthode de Kutta -Merson

```
% rkm(@equadif,[ti tf dt],[y1i y2i y3i], options)
% methode de Kutta-Merson explicite ordre 5
%

5 function [tvec, yvec]=rkm(f, param, y0, options)
  AbsTol=1e-6;
  % intervalle d'etude en temps et pas
  t0 = param(1);
  tf = param(2);
10  h = param(3);

  % sauvegarde du vecteur y0 dans la 1ere ligne du tableau yvec
  n=1;
  tvec(n) = t0;
15  yvec(n, :) = y0;

  t = t0;
  y = y0;

20 while (t<tf)
    k1=f(t, y);
    k2=f(t+h/3., y+h/3.*k1);
    k3=f(t+h/3., y+h/6.*k1+h/6.*k2);
    k4=f(t+h/2., y+h/8.*k1+3.*h/8.*k3);
25    k5=f(t+h, y+h/2.*k1-3./2.*h*k3+2.*h*k4);

    A1=y+h/2.*(k1-3.*k3+4.*k4);
    A2=y+h/6.*(k1+4.*k4+k5);

30    E=(A1-A2)/5.;
    norm_E=norm(E, inf);
    h=0.9*(AbsTol/norm_E)^0.2*h;

    if (norm_E <= AbsTol)
35        % pas suivant
        t=t+h;
        n=n+1;

        y=A2-E;
40    % stockage
        tvec(n)=t;
        yvec(n, :)=y;
    end;
end
```